

Dokumentacja Techniczna i Projektowa: Embedded Bird Classifier

Autorzy projektu: Emil Siatka, Mateusz Szwagierczak
Data wydania: Czerwiec 2026 r.
Wersja systemu: 1.0.0
Platforma bazowa: Raspberry Pi 4 Model B (2GB RAM)
System operacyjny: Raspberry Pi OS (64-bit, Debian Bookworm based)

1. Wstęp i Cel Projektu

Projekt **Embedded Bird Classifier** to autonomiczne, zintegrowane urządzenie klasy embedded przeznaczone do długotrwałego, terenowego monitoringu akustycznego awifauny (ptaków). System został zaprojektowany z myślą o pracy w trudnych warunkach środowiskowych (tzw. „tryb leśny”), bez stałego dostępu do infrastruktury sieciowej oraz zewnętrznego zasilania sieciowego.

Główne założenia funkcjonalne:

- Autonomiczny nasłuch ciągły:** Cykliczne nagrywanie próbek dźwiękowych z otoczenia za pomocą dedykowanego toru audio.
- Lokalna analiza AI (Edge Computing):** Przetwarzanie i klasyfikacja nagrań w czasie rzeczywistym bezpośrednio na urządzeniu za pomocą algorytmów głębokiego uczenia maszynowego (**BirdNet**).
- Izolowana dystrybucja danych w terenie:** Emisja własnego punktu dostępowego Wi-Fi (Hotspot), umożliwiającą bezprzewodowe pobieranie danych i podgląd wyników na urządzeniach mobilnych lub komputerach bez użycia internetu.
- Bezpieczna gospodarka pamięcią:** Agregacja pociętych próbek audio oraz logów w ustrukturyzowanych plikach JSON z możliwością exports do skompresowanego archiwum ZIP i zdalnego czyszczenia pamięci flash.

2. Specyfikacja i Architektura Sprzętowa (Hardware)

Urządzenie charakteryzuje się budową modułową, opartą na komponentach COTS (Commercial Off-The-Shelf), co zapewnia powtarzalność konstrukcji oraz łatwość serwisowania.

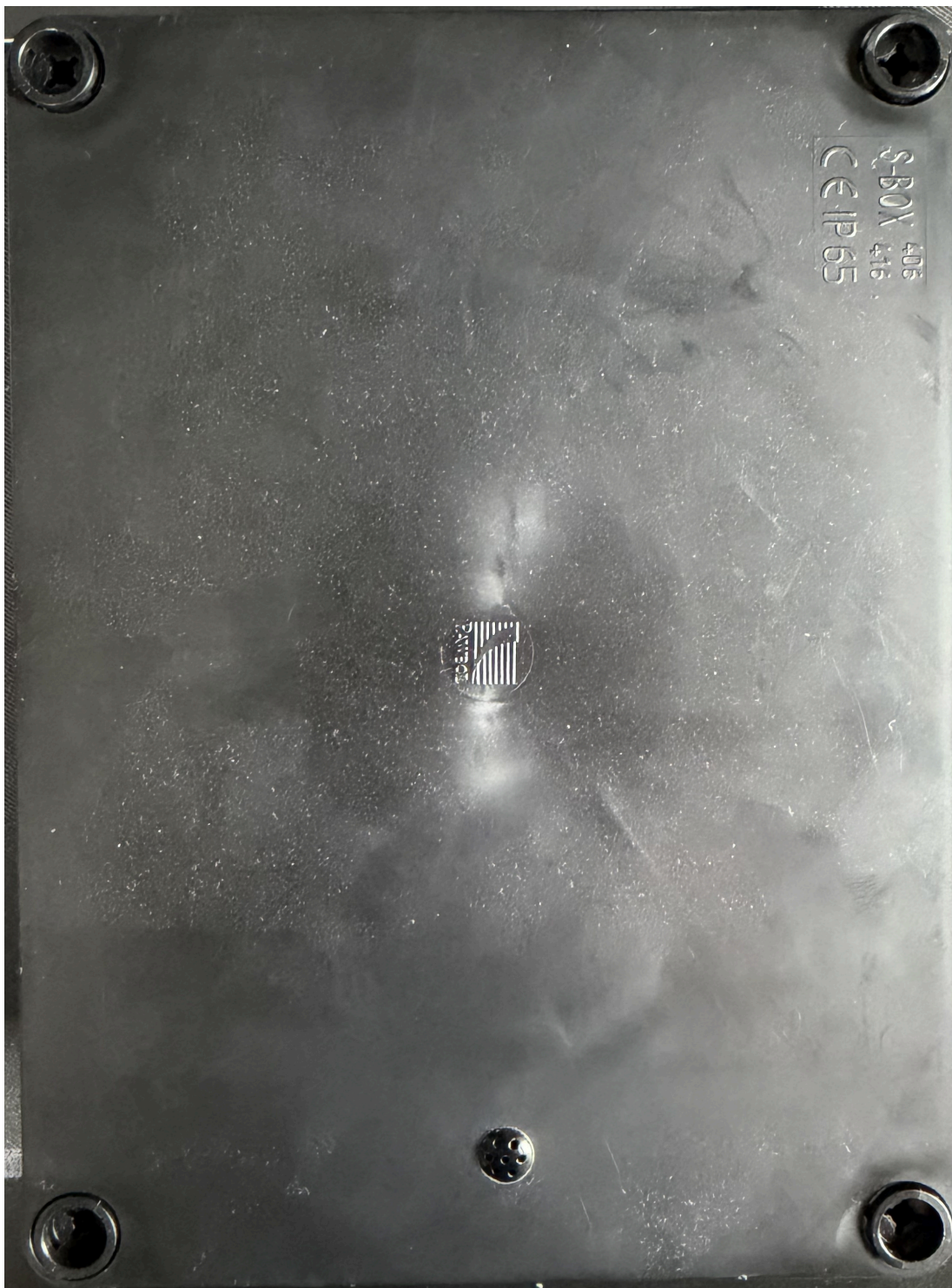
Komponent	Model / Producent	Rola w systemie	Specyfikacja techniczna / Uwagi
Jednostka centralna	Raspberry Pi 4 Model B	Główny procesor obliczeniowy, obsługa systemu operacyjnego, analiza AI, serwer API/WWW.	Broadcom BCM2711 (Quad-core Cortex-A72 @1.5GHz), 2GB pamięci RAM LPDDR4, wbudowany moduł Wi-Fi 2.4/5.0 GHz.
Układ zasilania	Powerbank Xiaomi	Autonomiczne źródło energii, funkcja bufora UPS.	Wsparcie dla technologii <i>pass-through charging</i> (jednoczesne

Komponent	Model / Producent	Rola w systemie	Specyfikacja techniczna / Uwagi
			Ładowanie akumulatora i zasilanie minikomputera z zewnętrznego źródła).
Interfejs audio input	Mikrofon krawatowy Esperanza	Przechwytywanie sygnałów akustycznych z otoczenia.	Charakterystyka wielokierunkowa, pasmo przenoszenia dopasowane do rejestracji dźwięków natury.
Przetwornik ADC	Karta dźwiękowa USB LogiLink	Cyfryzacja sygnału analogowego z mikrofonu.	Chipset obsługujący standard Plug&Play w systemie Linux, dedykowane wejście mikrofonowe TRS 3.5 mm.
Obudowa zewnętrzna	Puszka elektroinstalacyjna S-BOX (Pawbol)	Ochrona komponentów przed warunkami atmosferycznymi.	Klasa szczelności IP65, wykonana z polipropylenu o wysokiej wytrzymałości mechanicznej.

Integracja fizyczna i uszczelnienie:

Komponenty wewnątrz puszki S-BOX zostały unieruchomione przy użyciu kabli zasilających oraz nylonowych opasek uciskowych (trytek), co zabezpiecza je przed uszkodzeniami mechanicznymi podczas transportu. Tor audio został wyprowadzony na zewnątrz obudowy przez fabryczny przepust kablowy. Miejsce wyprowadzenia mikrofonu oraz newralgiczne połączenia strukturalne zostały zalane klejem termotopliwym (na gorąco), zapewniając doskonałą izolację od wilgoci i deszczu. Na zewnątrz puszki mikrofon osłonięto dedykowaną, perforowaną maskownicą ochronną.

Dokumentacja wizualna komponentów sprzętowych:



Rysunek 1: Autonomiczna kapsuła pomiarowa w osłonie S-BOX z wyprowadzonym i zabezpieczonym mikrofonem w dolnej części.



Rysunek 2: Rozmieszczenie komponentów wewnątrz puszek elektroinstalacyjnej. Widoczne zabezpieczenie Raspberry Pi 4, powerbanka buforowego oraz miniaturowej karty dźwiękowej USB LogiLink.

3. Architektura Oprogramowania i Stos Technologiczny

System działa w architekturze wieloprocesowej, składającej się z niezależnych, lecz ściśle współpracujących usług systemowych oraz modułów aplikacyjnych.

Stos Technologiczny (Tech Stack):

- **System Operacyjny:** Raspberry Pi OS (64-bit), zoptymalizowany pod kątem wyłączenia podsystemów oszczędzania energii (całkowita blokada acpi/sleep poprzez `systemctl mask`).
- **Backend:** Python 3.11+ osadzony w izolowanym środowisku wirtualnym (`venv`). Kluczowe biblioteki: `FastAPI` (serwer REST API), `uvicorn` (serwer ASGI), `birdnetlib` (lokalna inferencja modelu TensorFlow Lite), `watchdog` (reaktywne monitorowanie zdarzeń systemu plików), `sounddevice` i `scipy` (zarządzanie buforem audio i zapis WAV), `pydub` (manipulacja plikami audio).
- **Frontend:** Single Page Application (SPA) zbudowana w oparciu o framework `React` oraz narzędzie budowania `Vite`. Stylizacja zrealizowana za pomocą `TailwindCSS` z zestawem ikon `Lucide React` oraz komponentami wykresów dynamicznych `Recharts`.

Struktura Katalogów Projektu:

```
/home/user/bird_classifier/  
├─ autorun_logs.md          # Logi diagnostyczne autostartu  
├─ bird_images/             # Lokalna baza obrazów gatunków ptaków do UI  
├─ frontend/                # Kod źródłowy aplikacji React (Vite)  
│   ├─ index.html  
│   ├─ package.json  
│   └─ tailwind.config.js
```

```

├── src/
│   ├── App.jsx           # Główny interfejs dashboardu
│   ├── main.jsx
│   └── index.css
├── frontend.log          # Logi wyjściowe serwera deweloperskiego Vite
├── Raspberry_config-PL.md # Podstawowa instrukcja wdrożeniowa
├── requirements.txt      # Zależności pip dla Pythona
├── run.sh                # Główny skrypt orkiestracyjny procesami
├── setup.sh              # Skrypt instalacyjno-konfiguracyjny
├── src/                  # Skrypty backendowe (Core)
│   ├── __init__.py
│   └── analyzer.py      # Proces klasyfikacji AI i detekcji zmian
plików
├── log_reader.py         # Moduł segmentacji audio (pydub)
├── recorder.py           # Proces ciągłego nagrywania z mikrofonu
├── server.py             # Główny serwer FastAPI + serwowanie statyczne
├── running/              # Dynamiczny katalog roboczy systemu (Runtime)
│   ├── export.zip        # Tymczasowa paczka eksportowa danych
│   ├── analyzing_results/ # Pliki raportów sesji w formacie JSON
│   ├── new_audio_samples/ # Bufor wejściowy dla surowych plików WAV (9s)
│   └── saved_audio_samples/ # Chronologiczne podkatalogi z pociętymi
próbkami

```

Poprawność wdrożenia struktury plików oraz uprawnienia wykonawcze skryptów powłoki systemowej weryfikuje listing plików w systemie Linux:

```

user@RaspberrySW:~$ ls -l bird_classifier/
total 88
-rwxrwxrwx 1 user user 4339 Apr 21 02:25 autorun_logs.md
drwxrwxrwx 2 user user 12288 May 27 18:33 bird_images
drwxrwxrwx 4 user user 4096 Jun 10 15:59 frontend
-rwxrwxrwx 1 user user 172 Jun 10 15:59 frontend.log
-rwxrwxrwx 1 user user 1085 May 27 18:27 LICENSE
-rwxrwxrwx 1 user user 6334 May 27 18:27 Raspberry_config.md
-rwxrwxrwx 1 user user 6551 May 27 18:27 Raspberry_config-PL.md
-rwxrwxrwx 1 user user 26 May 27 18:27 README.md
-rwxrwxrwx 1 user user 1304 May 27 18:27 requirements.txt
drwxrwxrwx 5 user user 4096 May 28 13:47 running
-rwxrwxrwx 1 user user 1826 May 27 20:09 run.sh
-rwxrwxrwx 1 user user 1767 May 27 18:27 setup.sh
drwxrwxrwx 3 user user 4096 May 27 19:58 src
drwxrwxrwx 6 user user 4096 May 27 20:47 venv
user@RaspberrySW:~$

```

Rysunek 3: Sprawdzenie struktury środowiska operacyjnego i flag wykonywalności (+x) dla run.sh oraz setup.sh.

4. Przepływ Danych w Systemie (Data Flow)

System realizuje zamkniętą pętlę przetwarzania danych bez udziału człowieka:

```

[ Otoczenie ]
| (Dźwięk analogowy)

```



- Rejestracja Sygnału:** Moduł `recorder.py` otwiera strumień wejściowy na zdefiniowanym urządzeniu audio (karta dźwiękowa LogiLink). Próbkuje sygnał z częstotliwością 48 kHz w trybie mono. Co 9 sekund zawartość bufora jest zapisywana do pliku `audio_[TIMESTAMP].wav` w katalogu tymczasowym `new_audio_samples`. W konsoli generowany jest pseudograficzny wskaźnik wysterowania (VU-meter) obrazujący poziom głośności.
- Detekcja Nowej Próbkki:** Skrypt `analyzer.py` wykorzystuje mechanizm `watchdog` do monitorowania katalogu `new_audio_samples`. Wykrycie zdarzenia utworczego (`on_created`) dla pliku `.wav` wstrzymuje wątek na 1 sekundę (bufor zapobiegający wyścigowi danych na wolnych kartach pamięci), po czym przesyła plik do silnika analizy.
- Klasyfikacja AI:** Silnik `BirdNetLib` inicjalizuje lokalny model TensorFlow Lite, przekazując mu współrzędne geograficzne (domyślnie Warszawa: lat=52.2, lon=21.0) w celu zwiększenia trafności predykcji poprzez odfiltrowanie gatunków niewystępujących w danej strefie biogeograficznej.
- Utrwalanie i Segmentacja:** Jeśli model wykryje ptaka z wymaganim progiem pewności, ścieżka do pliku oraz pełna tablica detekcji trafiają do procesu `analyzer.py`. Następuje otwarcie lub dopisanie rekordu do pliku `analysis_[SESJA].json`. Równolegle moduł `log_reader.py` (wykorzystując `pydub`) wycina z 9-sekundowego pliku bazowego dokładny segment czasowy (`start_time` do `end_time`), w którym model zidentyfikował głos, i eksportuje go jako mały plik `.wav` do struktury chronologicznej.

Wynikowa struktura pliku JSON gromadzi zagnieżdżone detekcje wraz z pełną taksonomią dostarczaną przez platformę BirdNet:

```

[timestamp: "16:15:00",
 "file": "audio_20260610_161500.wav",
 "detections": [
   {
     "common_name": "House Sparrow",
     "scientific_name": "Passer domesticus",
     "start_time": 0.0,
     "end_time": 9.0,
     "confidence": 0.73762120689331,
     "label": "Passer domesticus_House Sparrow"
   }
 ]
}

[timestamp: "16:15:00",
 "file": "audio_20260610_161504.wav",
 "detections": [
   {
     "common_name": "House Sparrow",
     "scientific_name": "Passer domesticus",
     "start_time": 0.0,
     "end_time": 1.0,
     "confidence": 0.537884254550664,
     "label": "Passer domesticus_House Sparrow"
   },
   {
     "common_name": "House Sparrow",
     "scientific_name": "Passer domesticus",
     "start_time": 3.0,
     "end_time": 6.0,
     "confidence": 0.872389554977437,
     "label": "Passer domesticus_House Sparrow"
   },
   {
     "common_name": "House Sparrow",
     "scientific_name": "Passer domesticus",
     "start_time": 6.0,
     "end_time": 9.0,
     "confidence": 0.315529188209964,
     "label": "Passer domesticus_House Sparrow"
   }
 ]
}

[timestamp: "16:15:14",
 "file": "audio_20260610_161503.wav",
 "detections": [
   {
     "common_name": "House Sparrow",
     "scientific_name": "Passer domesticus",
     "start_time": 0.0,
     "end_time": 1.0,
     "confidence": 0.8279625177383423,
     "label": "Passer domesticus_House Sparrow"
   },
   {
     "common_name": "House Sparrow",
     "scientific_name": "Passer domesticus",
     "start_time": 3.0,
     "end_time": 6.0,
     "confidence": 0.7682893795348842,
     "label": "Passer domesticus_House Sparrow"
   },
   {
     "common_name": "House Sparrow",
     "scientific_name": "Passer domesticus",
     "start_time": 6.0,
     "end_time": 9.0,
     "confidence": 0.696018551580958,
     "label": "Passer domesticus_House Sparrow"
   }
 ]
}
]

```

Rysunek 4: Podgląd struktury kluczy JSON przesyłanych do frontendu, zawierających ramy czasowe oraz prawdopodobieństwo predykcji (confidence).

- Konsumpcja Danych:** Serwer FastAPI udostępnia endpointy REST oraz montuje statyczne punkty dostępu do plików. Aplikacja frontendowa w React odpytuje serwer w interwale 5-sekundowym (Live-Reload), pobierając najświeższe dane o wykryciach i renderując wykresy oraz interaktywny odtwarzacz audio.

5. Implementacja Backend (Python)

5.1. `src/recorder.py`

Proces odpowiedzialny za niskopoziomowe operacje wejścia/wyjścia na urządzeniu audio. Implementuje automatyczne wyszukiwanie mikrofonu na magistrali USB.

```

# Kluczowy fragment mechanizmu rejestracji bufora audio
def record(device_id):
    samplerate = 48000
    channels = 1
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    file_path = os.path.join(FOLDER_AUDIO, f"audio_{timestamp}.wav")
    recording_buffer = []

    def callback(indata, frames, time_info, status):
        # Obliczanie poziomu wysterowania i renderowanie paska VU w konsoli
        volume_norm = np.linalg.norm(indata) * 20 / np.sqrt(len(indata))
        recording_buffer.append(indata.copy())
        level = min(int(volume_norm * BAR_LENGTH), BAR_LENGTH)
        bar = "█" * level + "-" * (BAR_LENGTH - level)
        print(f"\rRecording (ID:{device_id}): [{bar}]", end="", flush=True)

    with sd.InputStream(samplerate=samplerate, channels=channels,
        callback=callback, device=device_id):
        time.sleep(RECORDING_DURATION)

```

```
full_recording = np.concatenate(recording_buffer, axis=0)
write(file_path, samplerate, full_recording)
```

5.2. src/analyzer.py

Rdzeń analityczny systemu. Działa w oparciu o architekturę sterowaną zdarzeniami (Event-Driven Architecture).

```
class BirdWatchHandler(FileSystemEventHandler):
    def on_created(self, event):
        if event.is_directory or not event.src_path.lower().endswith(".wav"):
            return

        time.sleep(1) # Zabezpieczenie IO przed niepełnym zapisem pliku
        try:
            recording = Recording(analyzer, event.src_path, lat=52.2, lon=21.0)
            recording.analyze()

            if recording.detections:
                # Blok operacyjny zapisu przyrostowego do pliku JSON
                if not os.path.exists(file_path):
                    with open(file_path, "w", encoding="utf-8") as f:
                        json.dump([], f)

                record_data = {
                    "timestamp": os.path.basename(event.src_path)[6:-4],
                    "file": os.path.basename(event.src_path),
                    "detections": recording.detections
                }

                with open(file_path, "r+", encoding="utf-8") as f:
                    data = json.load(f)
                    data.append(record_data)
                    f.seek(0)
                    json.dump(data, f, indent=4, ensure_ascii=False)
                    f.truncate()

                # Wywołanie zewnętrznego podmodułu ekstrakcji audio
                segment_audio_parts(recording.detections, event.src_path,
                                    SESSION_WAV_DIR, f"{os.path.basename(event.src_path)[: -4]}")
        except Exception as e:
            print(f"Błąd analizy: {e}")
```

5.3. src/server.py

Serwer FastAPI udostępniający interfejs programistyczny aplikacji (API) oraz realizujący operacje czyszczenia nośnika danych oraz budowania archiwum ZIP w locie.

- **GET /api/results**: Zwraca listę zarejestrowanych sesji (plików `.json`) posortowaną malejąco (najnowsze od góry).

- **GET /api/export:** Pakuje dynamicznie całą strukturę plików `analyzing_results` oraz folderów sesyjnych z `saved_audio_samples` do jednego zoptymalizowanego archiwum ZIP. Zapobiega to fragmentacji pobieranych danych.
- **DELETE /api/clear:** Dokonuje kaskadowego usunięcia wszystkich danych wynikowych z karty pamięci SD, przywracając system do stanu czystego (odpowiedź systemu widoczna na zrzutach ekranu `confirmation_deletion.png`).

6. Interfejs Użytkownika (Frontend - React)

Aplikacja kliencka realizuje wzorzec projektowy Dashboardu Operacyjnego w trybie Dark Mode.

Kluczowe mechanizmy zaimplementowane w `App.jsx`:

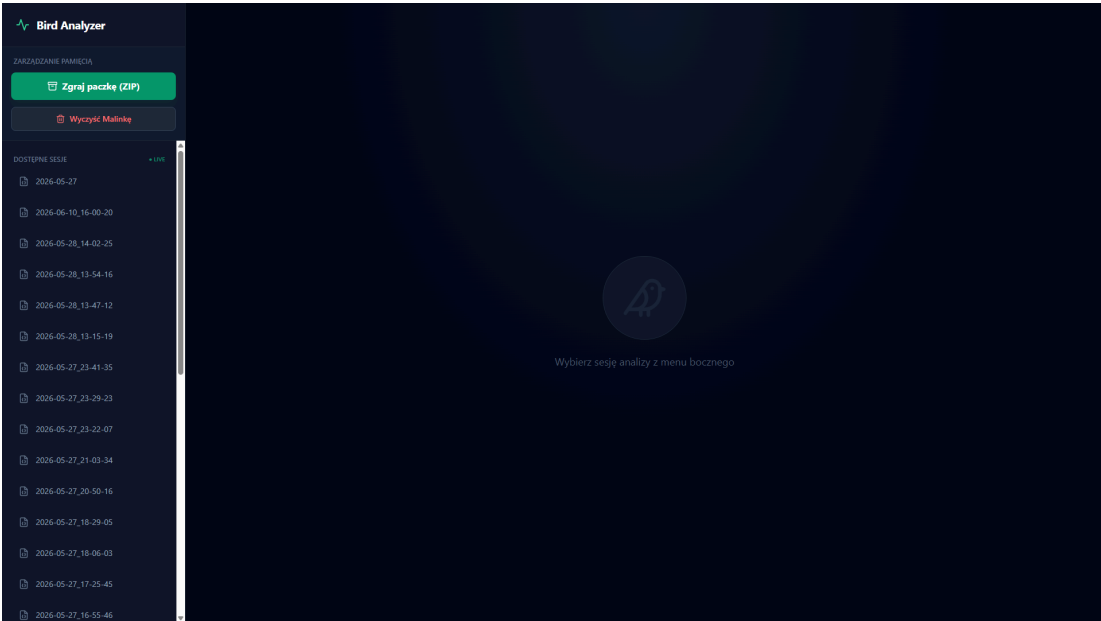
1. **Unikanie Stale Closures w Asynchronicznych Interwałach:** Zastosowano zaawansowany wzorzec z referencją `useRef` do przechowywania stanu aktywnego pliku:

```
const selectedFileRef = useRef(selectedFile);
selectedFileRef.current = selectedFile;
```

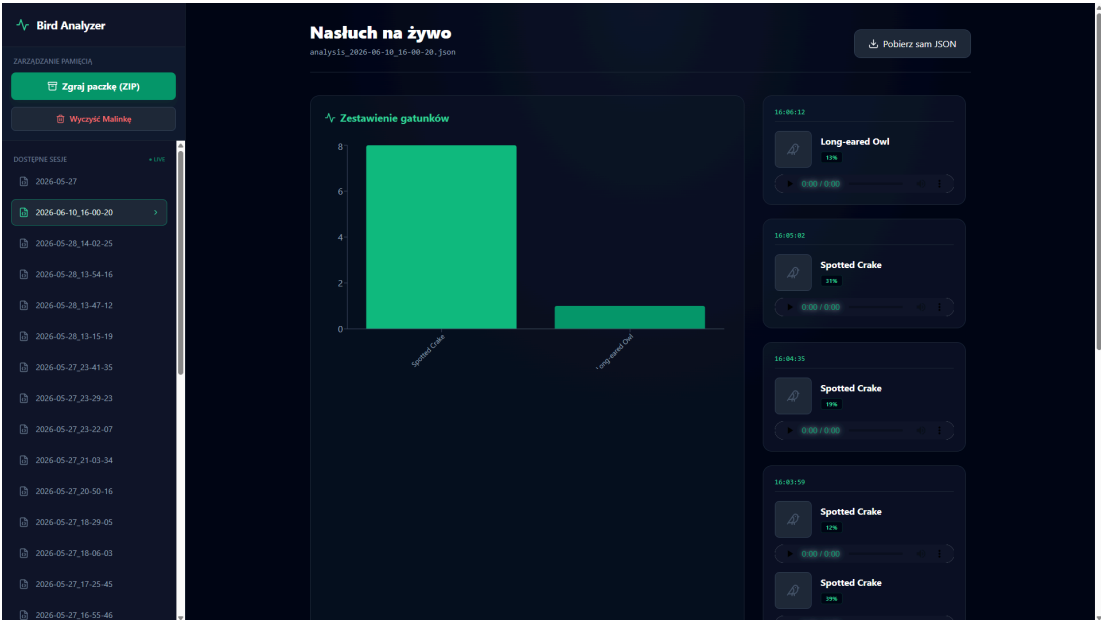
Dzięki temu uruchomiony proces `setInterval` (działający co 5 sekund) zawsze posiada dostęp do aktualnie wybranej przez użytkownika sesji, co pozwala na bezproblemowe odświeżanie wykresów w tle bez przerywania interakcji użytkownika (brak efektu migotania ekranu ładowania).

2. **Agregacja danych do wykresów:** Funkcja `generateChartData` mapuje zagnieżdżoną strukturę wykryć z formatu JSON na płaską strukturę asocjacyjną, zliczając wystąpienia unikalnych gatunków (`common_name`), a następnie sortuje wyniki malejąco. Wynik jest przekazywany do komponentu `<BarChart>` z biblioteki `recharts`.
3. **Obsługa Multimediiów i Cache-Busting:** Adresy URL próbek audio i raportów JSON są parametryzowane unikalnym znacznikiem czasu (`?t=${new Date().getTime()}`). Zapobiega to agresywnemu cache'owaniu plików przez przeglądarki mobilne (np. Google Chrome na Androidzie/iOS), wymuszając pobranie realnie zaktualizowanego pliku z serwera FastAPI podczas nasłuchu na żywo.

Prezentacja stanów interfejsu graficznego (UI):



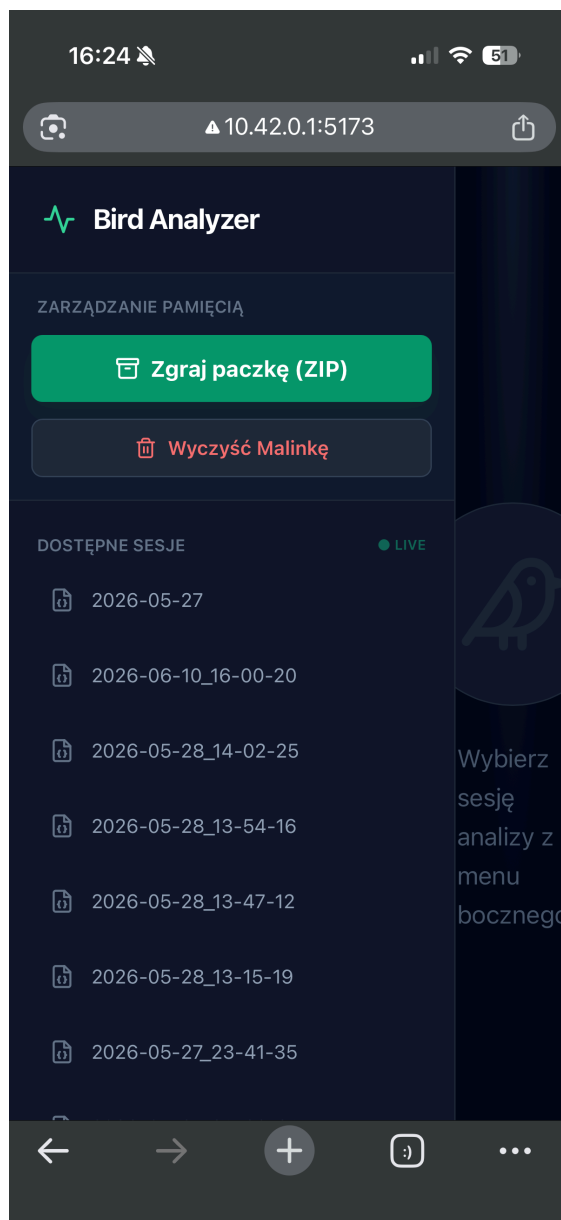
Rysunek 5: Widok panelu głównego po uruchomieniu urządzenia (oczekiwanie na wybór zarejestrowanej sesji).



Rysunek 6: Analiza wybranego pliku sesji. Po prawej stronie widoczny chronologiczny podgląd wykryć, a w części centralnej zagregowany wykres słupkowy dla gatunków Spotted Crane oraz Long-eared Owl.



Rysunek 7: Automatyczne mapowanie i dociąganie grafik z bazy lokalnej (przykład dla gatunku House Sparrow) w toku aktywnego napływu danych.



Rysunek 8: Responsywny widok bocznego menu zarządzania pamięcią oraz listy sesji wyświetlony bezpośrednio na ekranie

smartfona.

7. Środowisko Systemowe, Autostart i Wdrożenie Terenowe

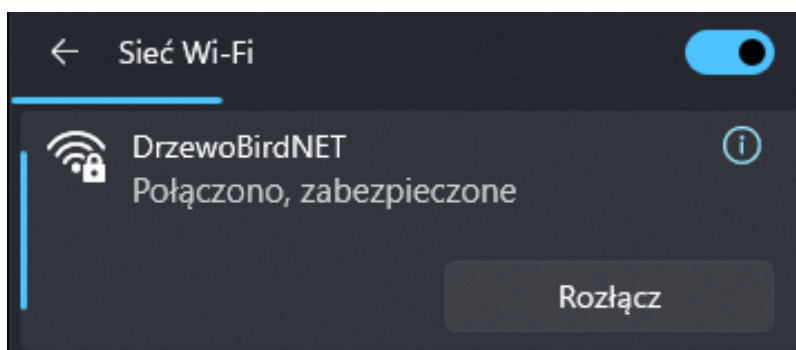
Urządzenie działa w pełni autonomicznie dzięki konfiguracji usług systemowych oraz menedżera sieci na poziomie warstwy jądra Linux.

7.1. Konfiguracja sieciowa (Terenowy Punkt Dostępowy)

Za pomocą narzędzia NetworkManager (`nmcli`) karta sieciowa `wlan0` zostaje przełączona w tryb AP (Access Point):

```
sudo nmcli device wifi hotspot ifname wlan0 ssid DrzewoBirdNET password
HasloDoDrzewa
sudo nmcli connection modify Hotspot connection.autoconnect yes
```

Konfiguracja ta wymusza na systemie przypisanie statycznego adresu IP bramy sieciowej: `10.42.0.1`.



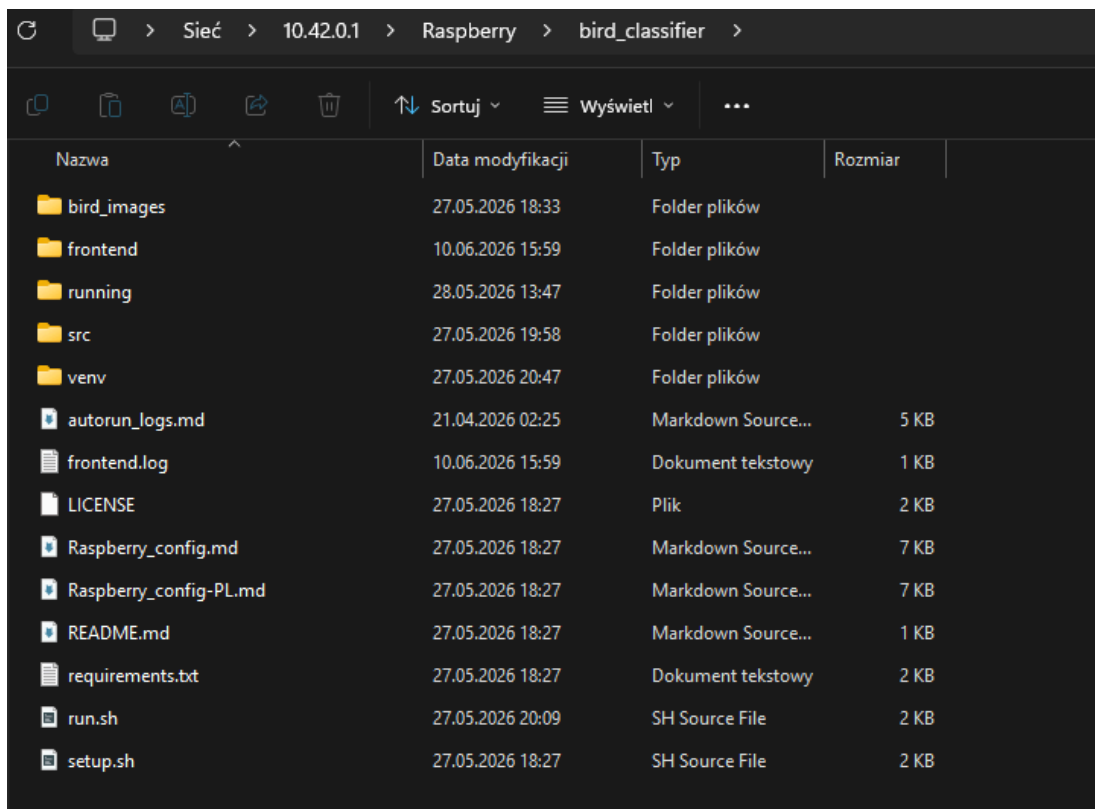
Rysunek 9: Kontrola ustanowionego połączenia Wi-Fi z terenową podsiecią badawczą „DrzewoBirdNET”.

7.2. Integracja z Windows (Serwer Samba)

W celu umożliwienia bezpośredniego zarządzania plikami z pominięciem interfejsu WWW, skonfigurowano demona Samba (`smbd`), który udostępnia katalog domowy użytkownika w sieci lokalnej. W sekcji `/etc/samba/smb.conf` zdefiniowano:

```
[Raspberry]
path = /home/user
writeable = yes
browseable = yes
public = no
create mask = 0644
directory mask = 0755
force user = user
```

Dostęp do systemu plików nośnika flash z poziomu systemu operacyjnego stacji klienckiej realizowany jest natywnie przez protokół SMB:



Rysunek 10: Mapowanie zasobów sieciowych i plików źródłowych projektu bezpośrednio w Eksploratorze Windows pod adresem IP bramy.

7.3. Orkiestracja i Autostart (Systemd)

Dedykowana usługa `birdnet.service` zapewnia automatyczne uruchomienie skryptu `run.sh` natychmiast po zainicjalizowaniu podsystemów audio i sieciowych przez jądro systemu, a także automatyczny restart w przypadku awarii jednego z procesów.

Plik konfiguracyjny usługi (`/etc/systemd/system/birdnet.service`):

```
[Unit]
Description=Embedded Bird Classifier
After=network.target sound.target

[Service]
Type=simple
User=user
WorkingDirectory=/home/user/bird_classifier
ExecStart=/bin/bash /home/user/bird_classifier/run.sh
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Podział na unikalne katalogi runtime dla każdej unikalnej pętli czasowej nagrywania przedstawia struktura drzewa procesów na dysku twardym:


```
user@RaspberrySW:~ $ tree -d bird_classifier/running/
bird_classifier/running/
├── analyzing_results
├── new_audio_samples
└── saved_audio_samples
    ├── 2026-05-27_19-58-16
    ├── 2026-05-27_20-00-21
    ├── 2026-05-27_20-06-28
    ├── 2026-05-27_20-10-10
    ├── 2026-05-27_20-12-54
    ├── 2026-05-27_20-52-54
    ├── 2026-05-27_20-53-05
    ├── 2026-05-27_20-53-06
    ├── 2026-05-27_20-53-47
    ├── 2026-05-27_20-53-49
    ├── 2026-05-27_21-03-34
    ├── 2026-05-27_21-23-56
    ├── 2026-05-27_21-28-24
    ├── 2026-05-27_21-28-16
    ├── 2026-05-27_21-31-25
    ├── 2026-05-27_22-42-22
    ├── 2026-05-27_22-49-00
    ├── 2026-05-27_22-49-47
    ├── 2026-05-27_22-52-18
    ├── 2026-05-27_22-53-07
    ├── 2026-05-27_23-22-07
    ├── 2026-05-27_23-25-55
    ├── 2026-05-27_23-29-23
    ├── 2026-05-27_23-41-35
    ├── 2026-05-28_13-15-19
    ├── 2026-05-28_13-40-28
    ├── 2026-05-28_13-43-00
    ├── 2026-05-28_13-47-12
    ├── 2026-05-28_13-54-16
    ├── 2026-05-28_14-02-25
    └── 2026-06-10_16-00-20

35 directories
user@RaspberrySW:~ $
```

Rysunek 11: Zrzut konsoli z polecenia `tree -d` wykazujący separację odizolowanych katalogów dla próbek audio z poszczególnych dat.

8. Instrukcja Obsługi i Procedury Terenowe

Krok 1: Uruchomienie i Połączenie

1. Podłącz system do źródła zasilania (Powerbank Xiaomi). Poczekaj ok. 40–60 sekund na pełne zainicjalizowanie procesów systemowych.
2. Na telefonie lub komputerze wyszukaj sieć Wi-Fi o nazwie **DrzewoBirdNET** i zaloguj się hasłem **HasloDoDrzewa** (zgodnie z załączonym zrzutem ekranu `connected_wifi.png` z rozdziału 7.1). Zignoruj komunikat systemowy o braku dostępu do internetu.

Krok 2: Monitorowanie danych (Trzy metody dostępu)

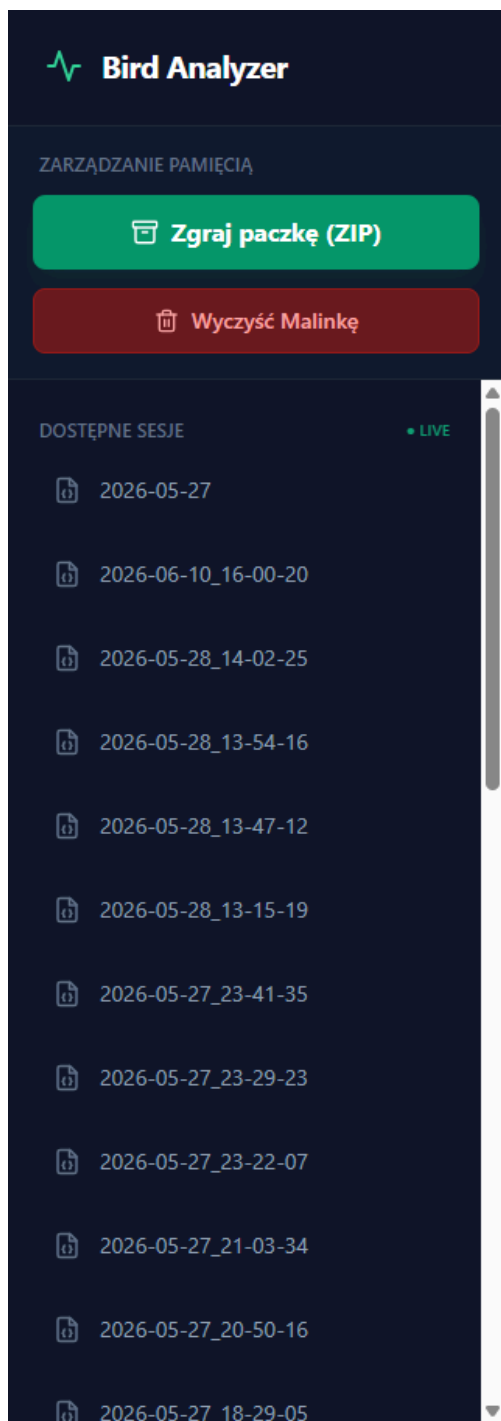
- **Aplikacja Webowa (Rekomendowana):** Otwórz dowolną przeglądarkę (Chrome, Safari, Edge) i przejdź pod adres <http://10.42.0.1:5173>. Otrzymasz pełny, responsywny dashboard (zgodnie z plikami `screens/birds_found_1.jpg`, `screens/birds_found_2.jpg` oraz mobilnym widokiem

screens/IMG_5700.jpg). Z menu bocznego wybierz żadaną sesję, aby przeglądać statystyki i odsłuchiwać pocięte próbki ptaków.

- **Eksplorator Windows:** Wpisz w pasek adresu menadżera plików: `\\10.42.0.1\Raspberry`. Po podaniu poświadczeń użytkownika `user`, uzyskasz natywny dostęp do struktury katalogów (widok screens/samba_windows.png).
- **Panel terminala (SSH):** W celu przeprowadzenia diagnostyki połącz się przez protokół SSH: `ssh user@10.42.0.1`.

Krok 3: Zgrywanie danych i konserwacja pamięci

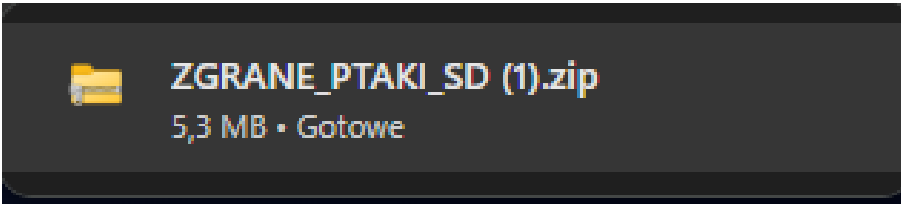
Zarządzanie pamięcią flash odbywa się w dedykowanej sekcji kontrolnej panelu bocznego:



Rysunek 12: Narzędzia administracyjne zapisu i usuwania logów zlokalizowane w lewym panelu dashboardu.

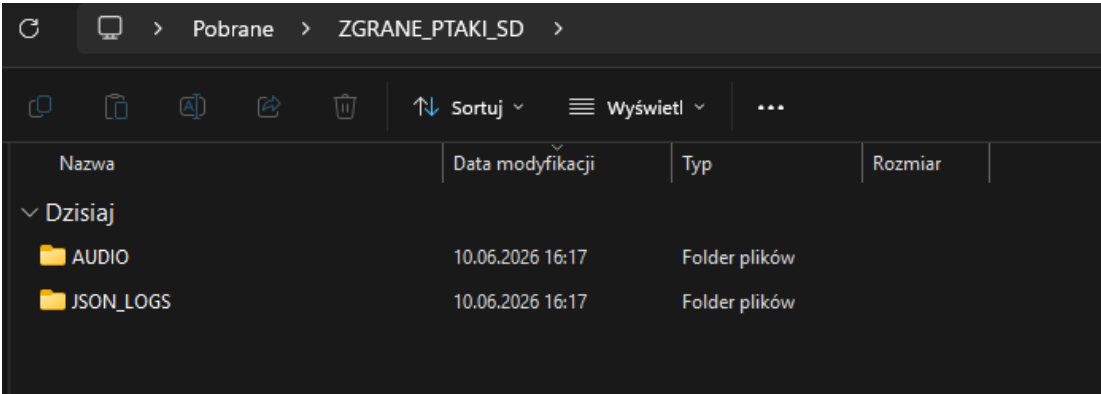
1. W panelu bocznym aplikacji kliknij zielony przycisk **"Zgraj paczkę (ZIP)"**.

2. System wygeneruje i pobierze skompresowane archiwum zbiorcze bezpośrednio na Twoje urządzenie:

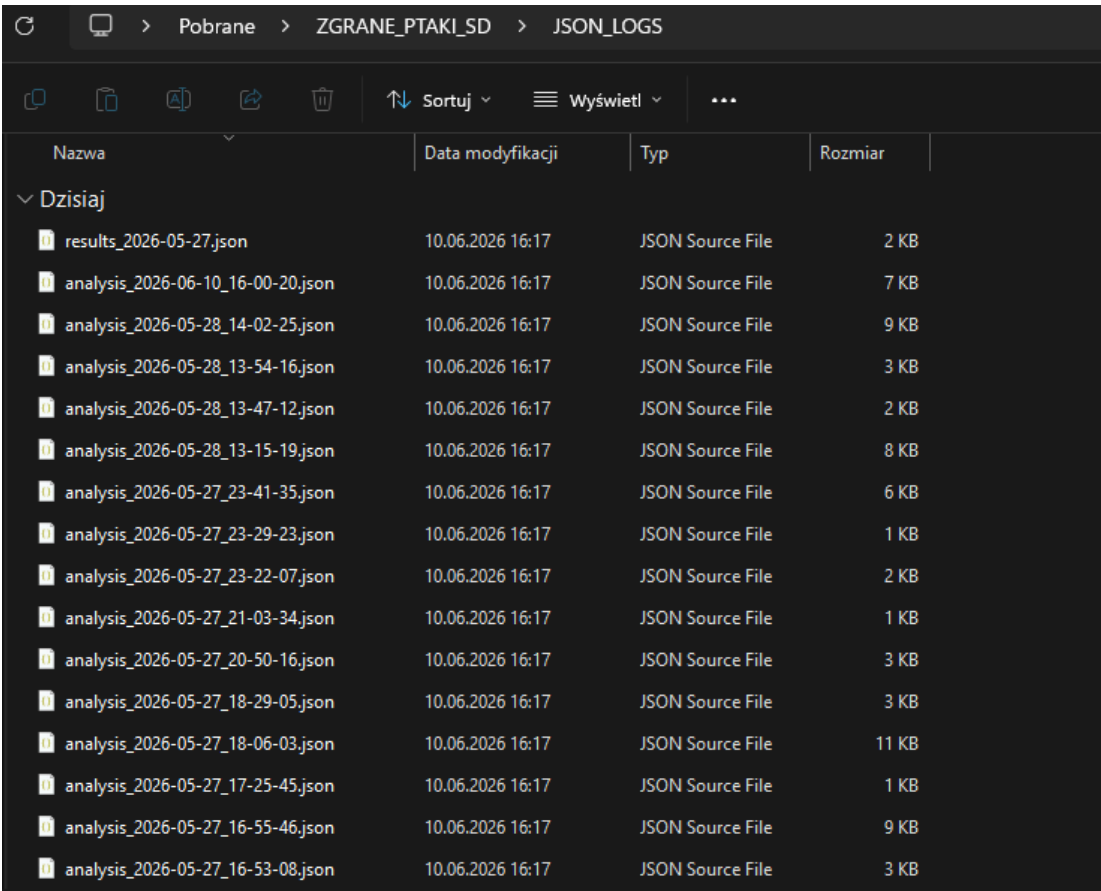


Rysunek 13: Monitor pobierania pliku archiwum ZGRANE_PTAKI_SD.zip o wadze 5.3 MB poprzez API systemu.

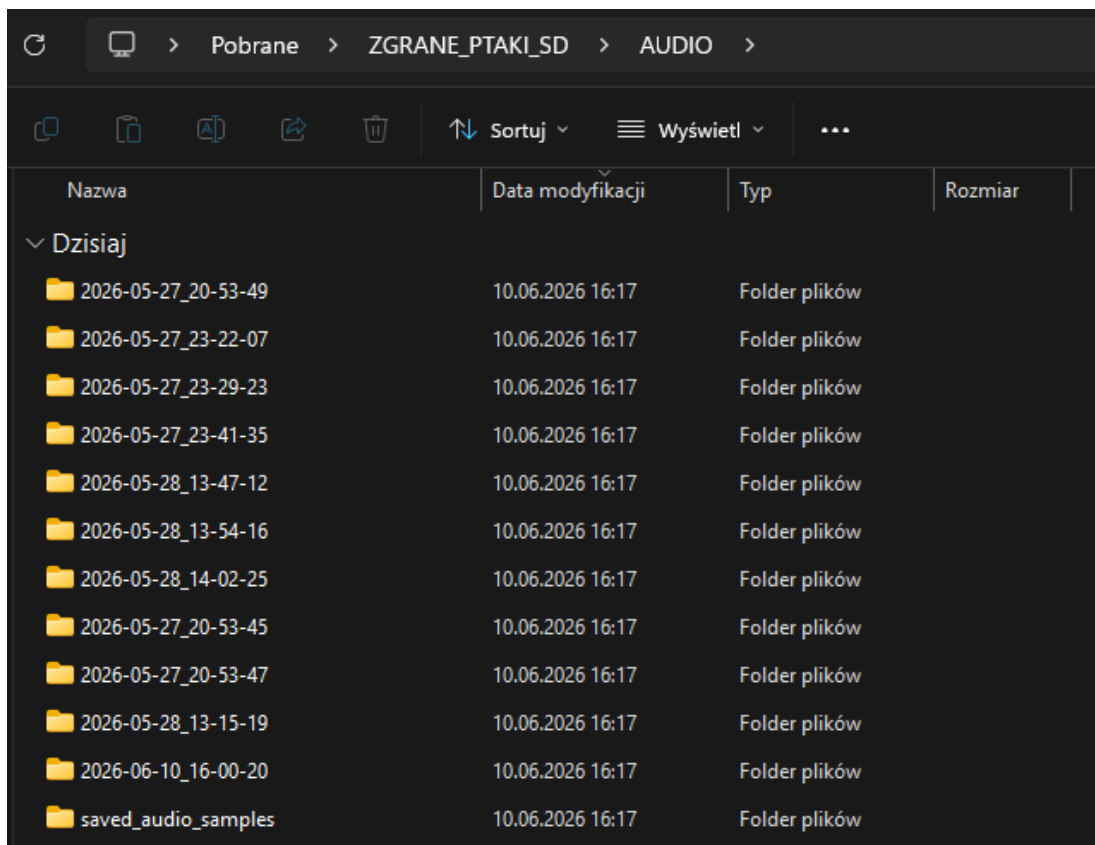
3. Układ rozpakowanego archiwum udostępnia ustrukturyzowany podział danych pomiarowych:



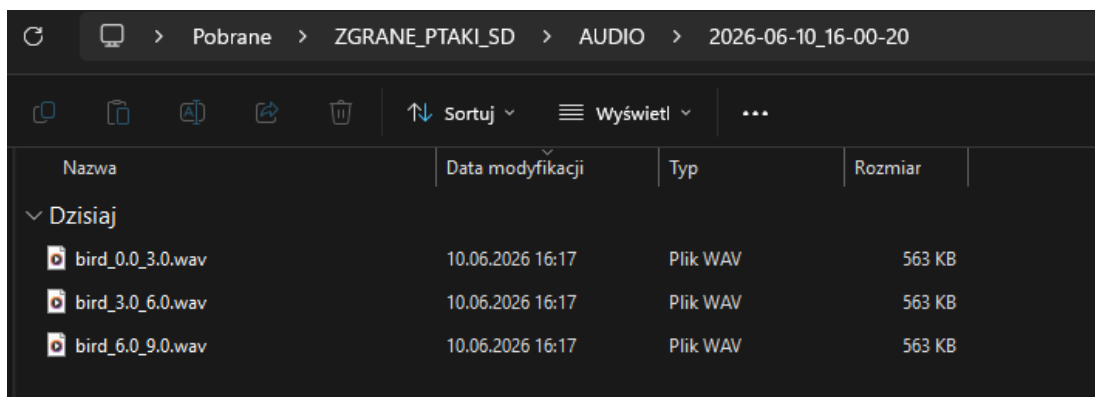
Rysunek 14: Główna przestrzeń wyeksportowanej paczki ZIP zawierająca katalogi AUDIO i JSON_LOGS.



Rysunek 15: Wszystkie chronologiczne pliki sesji tekstowych JSON poprawnie spakowane do folderu raportów.

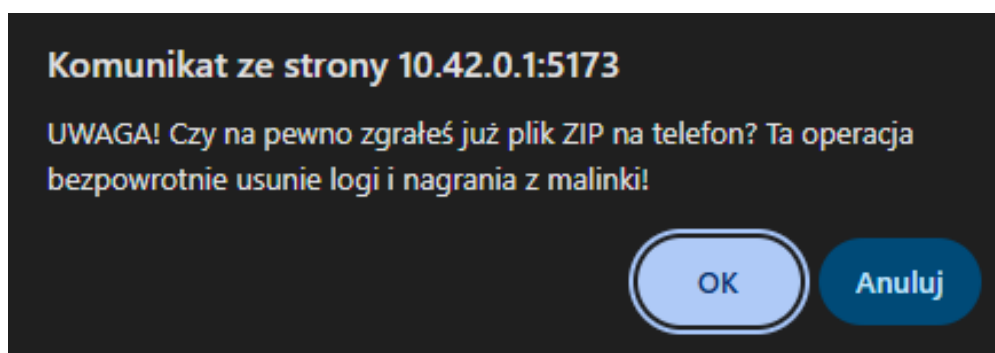


Rysunek 16: Katalogi próbek dźwiękowych posegregowane według sygnatury czasowej ich rejestracji w terenie.

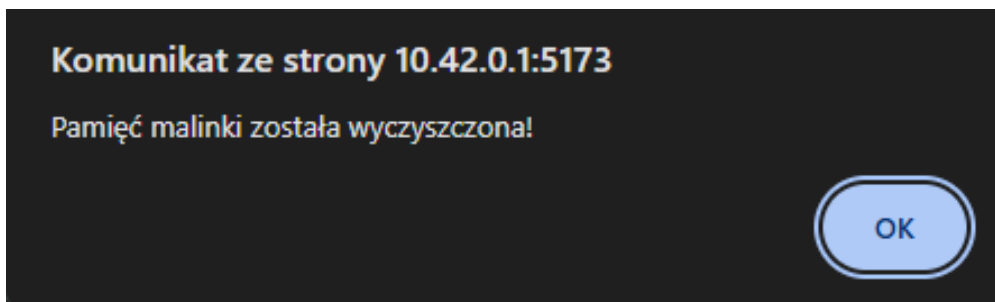


Rysunek 17: Pojedyncze pliki audio pocięte automatycznie przez funkcję segmentacji (log_reader.py) dla konkretnych wykrytych ptaków.

- Po upewnieniu się, że dane zostały w całości zarchiwizowane na Twoim urządzeniu klienckim (telefonie/komputerze), naciśnij czerwony przycisk **"Wyczyść Malinkę"**. Potwierdź chęć wykonania operacji w oknie dialogowym:

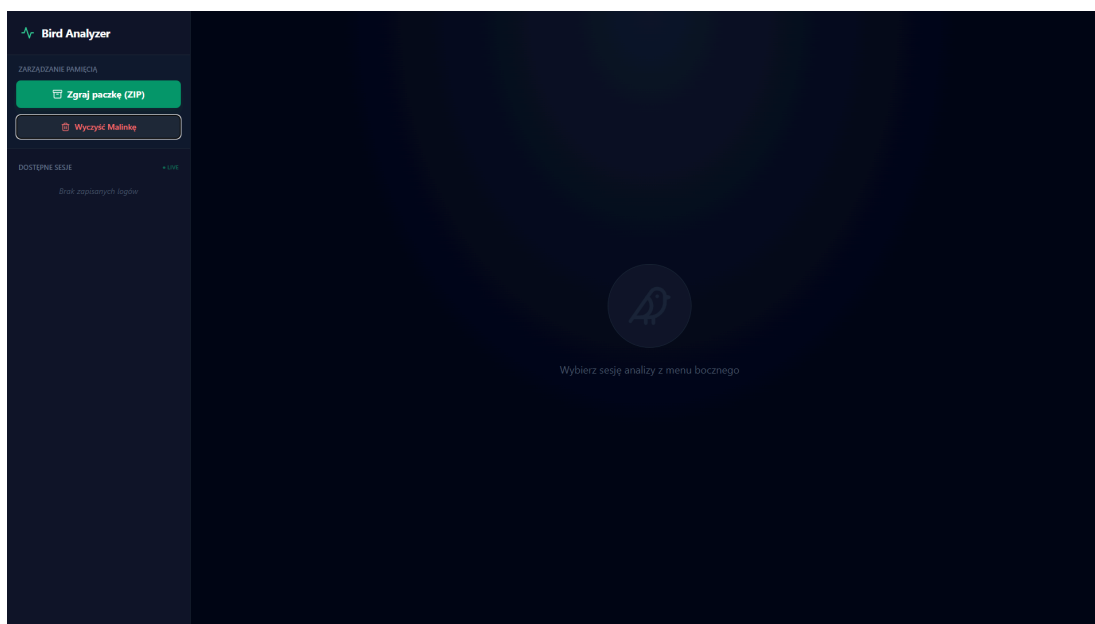


Rysunek 18: Systemowe zapytanie JavaScript zabezpieczające przed przypadkowym usunięciem nieskopiowanych danych w terenie.



Rysunek 19: Informacja zwrotna o udanym przebiegu kaskadowego czyszczenia zasobów.

5. Karta SD zostanie całkowicie zwolniona z danych bieżących, co przywraca aplikację oraz system plików minikomputera do czystego stanu wejściowego:



Rysunek 20: Dashboard operacyjny bezpośrednio po wyczyszczeniu bazy danych (stan czysty, brak dostępnych sesji).

```
user@RaspberrySW:~ $ tree -d bird_classifier/running/  
bird_classifier/running/  
├── analyzing_results  
├── new_audio_samples  
└── saved_audio_samples  
  
4 directories  
user@RaspberrySW:~ $
```

Rysunek 21: Weryfikacja terminalowa za pomocą `tree -d` potwierdzająca kaskadowe usunięcie dynamicznych podfolderów sesyjnych.